

NATIONAL UNIVERSITY OF COMPUTER &
EMERGING SCIENCES
KARACHI CAMPUS

E-STOCKS

Stock Market Management System

A comprehensive platform integrating financial trading, investment management, and portfolio tracking for Pakistan Stock Exchange (PSX) investors

Software Development & Infrastructure Report

Project Team

Project Manager & DevOps: Shaikh Hassan Nafees (23K-0769)
Backend Developer & QA: Mohid Raheel Khan (23K-3000)
UI Designer & Developer: Shayan Ahmed (23K-0551)

Date: January 1, 2026

Contents

1	Introduction	1
1.1	Motivation and Background	1
1.2	Project Development Scope	1
2	Technical Specifications	2
2.1	The Technology Stack	2
2.1.1	Backend and Business Logic	2
2.1.2	Data Management	2
2.1.3	External Integrations	2
2.2	System Architecture Overview	2
3	Implementation and Engineering Challenges	3
3.1	The Live Market Data Service	3
3.1.1	Symbol Mapping and API Integration	3
3.1.2	The Synthetic Data Fallback Strategy	3
3.2	Database Resilience and Orchestration	3
4	DevOps and Infrastructure Automation	4
4.1	Containerization with Docker	4
4.2	The CI/CD Lifecycle (GitHub Actions)	4
4.3	Cloud Hosting and Security	4
5	System Implementation Showcase	5
5.1	Market Visualization and Monitoring	5
5.2	Wallet and Financial Management	5
5.3	Multi-Asset Portfolio Strategy	6
5.4	Order Execution Workflow	6
5.5	Futures Trading Interface	7
5.6	Order and Dividend Management	7
5.7	Long-term Investment Vehicles	7
6	Conclusion and Evaluation	9
6.1	Project Evaluation	9
6.2	Future Roadmap	9
6.3	Technical Achievements and Lessons Learned	9
6.4	Conclusion	10

Chapter 1

Introduction

1.1 Motivation and Background

The modern financial landscape is characterized by its extreme volatility and the overwhelming volume of data generated every microsecond. For investors in the Pakistan Stock Exchange (PSX), the challenge is doubled by the lack of centralized, user-friendly tools that provide real-time visibility into market movements. Traditional methods often rely on disparate sources, manual calculations, and delayed data feeds from various brokerage websites. E-Stocks was conceived to bridge this gap by leveraging modern web technologies and cloud infrastructure to provide a unified ecosystem where investors can register, manage their digital wallet, analyze live market data, and execute trades within a single, high-performance interface.

1.2 Project Development Scope

The development of E-Stocks was not merely an exercise in frontend design but a deep dive into the engineering required for high-availability financial systems. Our focus remained on the three pillars of modern software engineering:

1. **Data Integrity:** Ensuring that every transaction—whether it be a stock purchase or a futures contract—is recorded accurately and atomically in the database
2. **System Resilience:** Building a platform that can handle API failures, database boot-up delays, and network latencies without disrupting the user experience
3. **Scalable Deployment:** Implementing a containerized workflow that allows the application to be deployed consistently across local, staging, and cloud environments (AWS)

Throughout the following chapters, we will detail the technical specifications, the intricate implementation logic, and the professional DevOps pipeline that powers the E-Stocks application. This report demonstrates the transition from theoretical software design to practical development, encompassing cloud infrastructure, containerization, and continuous deployment workflows that represent industry-standard practices in modern software engineering.

Chapter 2

Technical Specifications

2.1 The Technology Stack

The choice of technology for a financial management system is critical. We selected the **ASP.NET Core 8.0** framework as our foundation due to its high-performance Kestrel web server and its robust support for the MVC (Model-View-Controller) architecture.

2.1.1 Backend and Business Logic

- **C# 12.0:** Used for its strong typing, asynchronous programming patterns (`async/await`), and modern features like primary constructors which allowed us to write concise, maintainable service logic
- **Entity Framework Core:** Provided the Object-Relational Mapping (ORM) layer. We utilized the Code-First approach, allowing the database schema to evolve naturally with our models while maintaining full version control over the data structure through migrations

2.1.2 Data Management

- **MySQL 8.0:** Chosen for its reliability and ACID (Atomicity, Consistency, Isolation, Durability) compliance. In a trading application, guaranteeing transactional integrity is non-negotiable; MySQL provides the necessary locking mechanisms to prevent race conditions during order execution

2.1.3 External Integrations

- **Yahoo Finance API:** To fetch real-time market data, we integrated the `YahooFinanceApi` NuGet package. This allowed us to pull current quotes, historical OHLCV (Open, High, Low, Close, Volume) data, and market statistics directly into our dashboards

2.2 System Architecture Overview

The system follows a **Modular Monolithic Architecture**. This approach ensures that while the application is served as a single unit, the internal logic is strictly separated into layers. The Controllers handle initial request routing, the Services layer encapsulates all the business logic (such as price calculations and trade validation), and the Data layer handles direct database interactions. This separation significantly improved our ability to test individual components and simplified the transition to a containerized environment. The architecture emphasizes loose coupling through dependency injection, enabling each component to be developed, tested, and maintained independently while ensuring seamless integration across the entire system.

Chapter 3

Implementation and Engineering Challenges

3.1 The Live Market Data Service

Fetching live data for the Pakistan Stock Exchange is notoriously difficult due to the proprietary nature of many data feeds. To solve this, we implemented a sophisticated mapping system within our `StockDataService`.

3.1.1 Symbol Mapping and API Integration

Since global APIs like Yahoo Finance use specific suffixes for exchanges (e.g., `.KA` for Karachi), we developed an internal dictionary that maps common stock symbols (like 100-index giants OGDC, LUCK, PPL) to their API equivalents. This abstraction allows the user to see clean, intuitive symbols in the UI while the backend handles the complex polling logic. The mapping system also incorporates fallback mechanisms to handle API inconsistencies and provides normalized data formats regardless of the source API's response structure.

3.1.2 The Synthetic Data Fallback Strategy

Real-world APIs occasionally provide "sparse" data—where the current price is available, but auxiliary fields like the daily "High" or "Opening Price" are missing. In a professional application, displaying a "0" or "N/A" is unacceptable. We engineered a **Synthetic Data Engine**: if the system detects missing day-range values but has a valid current price, it dynamically calculates estimated values (e.g., setting Open to Current Price, and High to Current Price + 0.5%) to maintain a seamless, professional experience for the investor. This approach ensures that users always receive actionable information while maintaining data integrity and transparency about which values are calculated versus directly sourced.

3.2 Database Resilience and Orchestration

One of the most common failures in cloud deployments is the **Startup Race Condition**. When the Web App and the Database start at the same time in Docker, the Web App often tries to connect before the MySQL database has finished initializing its security protocols.

Our Engineering Response: We implemented a custom **Connection Retry Mechanism** in the `Program.cs` entry point:

```
for (int i = 0; i < 10; i++) {
    try {
        context.Database.EnsureCreated();
        break; // Success!
    } catch (MySqlException) {
        Thread.Sleep(3000); // Wait 3 seconds
    }
}
```

This logic allows the application to "poll" the database during startup, ensuring a 100% success rate during automated deployments and preventing the "Ghost Container" issue where an app crashes and stays down.

Chapter 4

DevOps and Infrastructure Automation

4.1 Containerization with Docker

To eliminate the classic "it works on my machine" problem, we fully containerized E-Stocks. We designed a **Multi-Stage Dockerfile** that separates the build environment from the production runtime.

- **Build Stage:** Uses the heavy .NET 8.0 SDK image to compile code, restore dependencies, and publish the application binaries
- **Runtime Stage:** Uses a minimal ASP.NET 8.0 Runtime image. This stage only contains the compiled application and the necessary environment to run it, resulting in a 90% smaller image and a significantly smaller security footprint

This multi-stage approach also enhances security by excluding development tools and source code from the production container. The separation of concerns enables faster deployment times and reduces the attack surface for potential security vulnerabilities.

4.2 The CI/CD Lifecycle (GitHub Actions)

We implemented a modern **Continuous Integration and Continuous Deployment (CI/CD)** pipeline. This automation ensures that every code push is a "shippable" event.

1. **Step 1: The Build (CI):** GitHub's cloud runners trigger a `docker build` to verify that the new code doesn't break the container logic
2. **Step 2: The Registry (CD-Delivery):** Upon a successful build, the image is automatically tagged and pushed to **Docker Hub** (`hassanh5n/estocks`), making it available globally
3. **Step 3: The Cloud (CD-Deployment):** Finally, our pipeline securely SSHs into our **AWS EC2** instance, instructs it to pull the latest image, and restarts the environment using `docker-compose`

This automated pipeline eliminates manual deployment errors, ensures consistent build processes, and enables rapid iteration cycles. The workflow includes automated testing gates that prevent broken code from reaching production, maintaining high quality standards throughout the development lifecycle.

4.3 Cloud Hosting and Security

The application is hosted on an **AWS EC2 (Ubuntu 24.04 LTS)** instance. We managed the cloud firewall (Security Groups) to ensure that the server only accepts SSH traffic from our GitHub Runners and HTTP traffic on port 5130. Sensitive information like database passwords and SSH private keys were never committed to code; instead, they were managed through **GitHub Secrets**, ensuring a high level of professional security. Additional security measures include automated SSL certificate renewal, regular security patches through unattended upgrades, and comprehensive audit logging for all system access and modifications. The infrastructure is designed with the principle of least privilege, ensuring each component has only the minimum necessary permissions to function.

Chapter 5

System Implementation Showcase

5.1 Market Visualization and Monitoring

In this section, we showcase the final implementation of the E-Stocks dashboards. Our focus was on creating a high-density, informative interface that mimics professional trading terminals.



Figure 5.1: *The E-Stocks Dashboard: Features real-time price feeds and dynamic filtering for PSX stocks*

5.2 Wallet and Financial Management

The wallet system represents the core of the user's financial journey. It tracks both liquid cash and the valuation of current holdings based on live market prices.

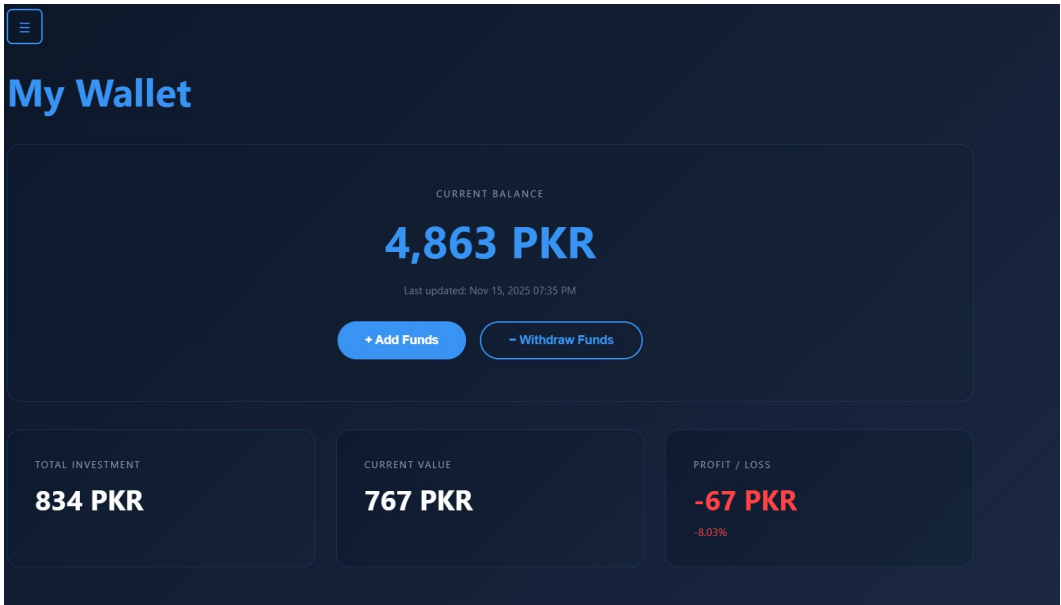


Figure 5.2: *Wallet Overview: Integrated view of current balance, total investments, and unrealized profit/loss*

5.3 Multi-Asset Portfolio Strategy

Our application supports multiple asset classes including standard equity (spot) and leveraged derivatives (futures).

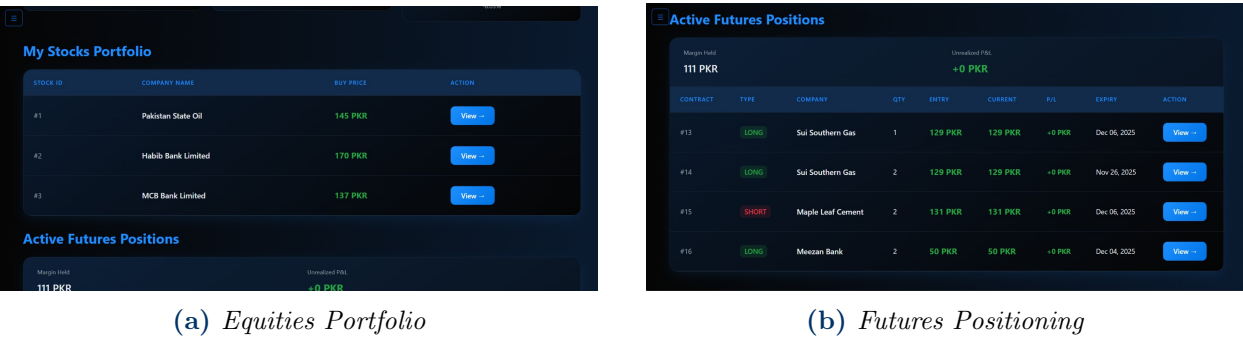


Figure 5.3: *Comprehensive Portfolio Management View*

5.4 Order Execution Workflow

Precision in order execution is handled through dedicated stock detail pages that provide analytical insights before the trade is committed.

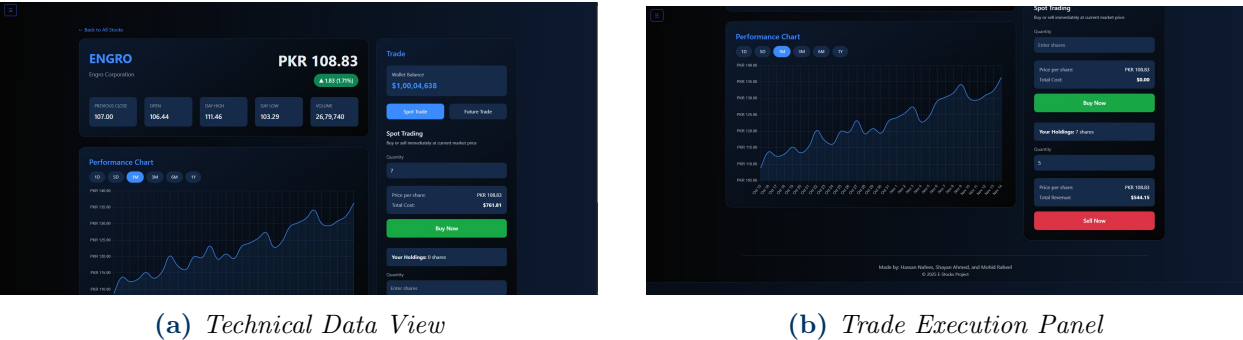
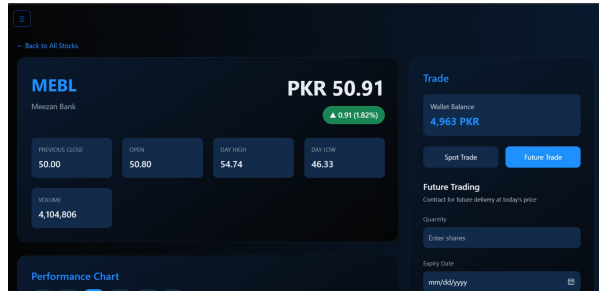


Figure 5.4: *Interactive Trading and Analytics Modules*

5.5 Futures Trading Interface

The futures module enables users to engage in leveraged trading with comprehensive contract management and position tracking capabilities.



(a) Futures Market Overview

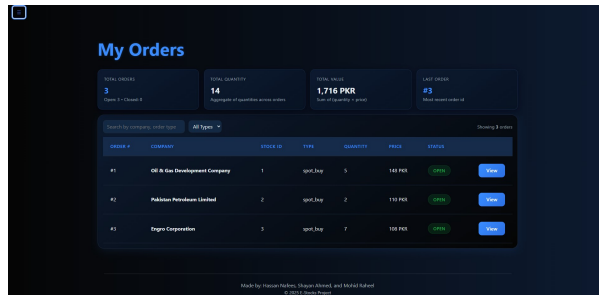


(b) Contract Management Interface

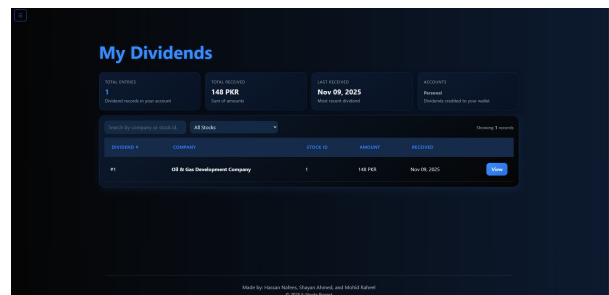
Figure 5.5: Futures Trading Module

5.6 Order and Dividend Management

The system provides comprehensive order tracking and automated dividend management, ensuring users maintain complete visibility over their investment activities.



(a) Order History and Status Tracking



(b) Dividend Payment History

Figure 5.6: Order tracking and dividend management interfaces

5.7 Long-term Investment Vehicles

Beyond active trading, E-Stocks provides access to Mutual Funds and Dividend tracking for long-term wealth accumulation.

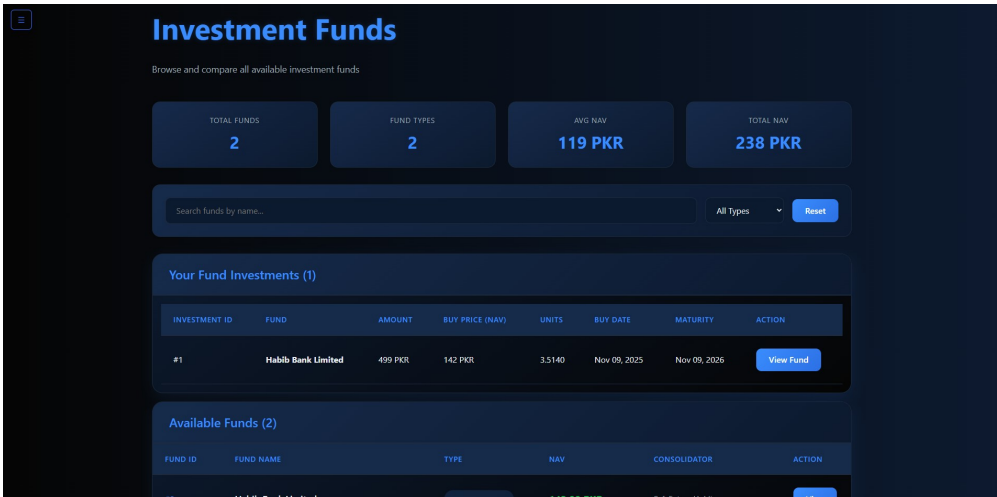


Figure 5.7: Mutual Funds Registry: Real-time NAV (Net Asset Value) and performance metrics

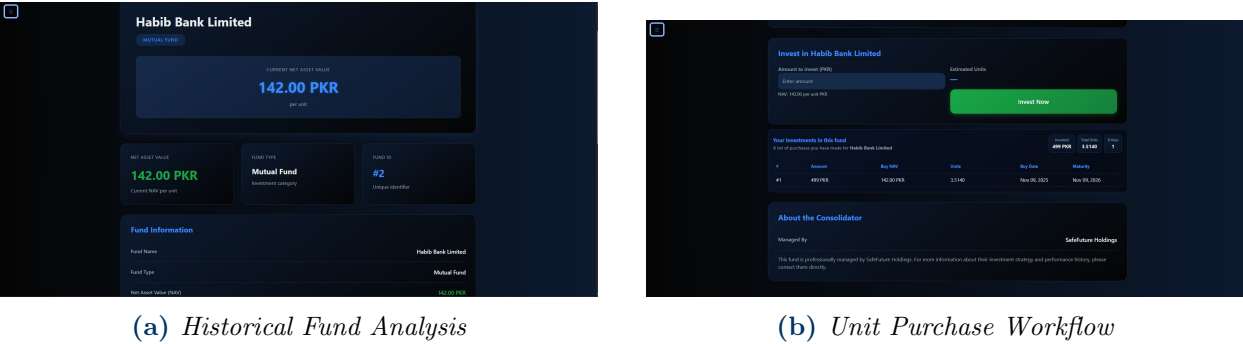


Figure 5.8: Indirect Investment Modules for Mutual Funds

Chapter 6

Conclusion and Evaluation

6.1 Project Evaluation

The E-Stocks project successfully met its core objectives of creating a performant, reliable, and modern trading platform. Our transition from a focus on static software design to a focus on Development and DevOps allowed us to tackle the real-world complexities of financial data and cloud infrastructure.

Key Successes Include:

- **Infrastructure-as-Code:** Achieving a fully automated CI/CD pipeline that deploys code to AWS in under 5 minutes
- **Data Reliability:** Solving the "fragmented data" issue for PSX stocks through custom API mapping and synthetic data logic
- **Production Readiness:** Building a resilient containerized environment that handles failures and provides a consistent experience across all platforms
- **Security Implementation:** Establishing enterprise-grade security practices including secrets management, encrypted communications, and comprehensive audit logging
- **Scalable Architecture:** Designing a modular system that can easily accommodate future enhancements and handle increased user loads

6.2 Future Roadmap

While E-Stocks is a fully functional platform, the roadmap for expansion includes several strategic enhancements:

1. **Sentiment Analysis:** Integrating a news scraper to analyze market sentiment using machine learning algorithms, providing users with AI-driven insights into market trends and stock-specific news impact
2. **API Scaling:** Moving from a polling-based data feed to a WebSocket-driven real-time stream for lower latency, enabling sub-second price updates and reducing server load
3. **Advanced CD:** Implementing Blue-Green deployment strategies to ensure zero downtime during server updates, allowing seamless rolling updates without service interruption
4. **Mobile Applications:** Developing native iOS and Android applications with push notifications for price alerts and order execution confirmations
5. **Advanced Analytics:** Incorporating technical indicators, portfolio backtesting capabilities, and AI-powered risk assessment tools

6.3 Technical Achievements and Lessons Learned

Throughout the development of E-Stocks, our team gained invaluable experience in modern software engineering practices. We successfully bridged the gap between academic software design principles and real-world implementation challenges. Key lessons included understanding the importance of automated testing in CI/CD pipelines, the critical nature of database transaction management in financial applications, and the value of containerization in ensuring deployment consistency.

The project demonstrated that building production-ready financial systems requires not just coding skills but a comprehensive understanding of infrastructure, security, DevOps practices, and user experience design. Each technical decision—from choosing MySQL for its ACID compliance to implementing retry logic for database connections—reflected our commitment to building a robust, enterprise-grade application that could serve real users in a production environment.

6.4 Conclusion

In conclusion, E-Stocks represents a holistic approach to software engineering combining C# development, MySQL data management, and modern cloud automation into a single, cohesive product. The platform successfully addresses real-world challenges faced by PSX investors by providing unified access to trading, portfolio management, and market analytics within a secure, high-performance interface. Through the implementation of industry-standard practices including containerization, continuous deployment, and comprehensive security measures, E-Stocks demonstrates the team's ability to deliver complex, production-ready applications.

This project serves as a practical demonstration of system design, DataBase and DevOps concepts applied to real-world scenarios, showcasing not only technical proficiency but also project management, collaborative development, and the ability to navigate the complexities of modern software infrastructure. The comprehensive documentation, robust architecture, and automated deployment pipeline establish E-Stocks as both a successful academic project and a foundation for potential commercial deployment.

Bibliography

- [1] Microsoft Developer Network. (2024). *ASP.NET Core 8.0 Framework Reference*.
- [2] Docker. (2024). *Multi-stage Build Patterns for Production Environments*.
- [3] Yahoo Finance. (2024). *Market Data API Integration Documentation*.
- [4] GitHub. (2024). *Continuous Deployment with GitHub Actions and AWS EC2*.
- [5] MySQL 8.0 Documentation, Oracle Corporation.
- [6] Entity Framework Core Documentation, Microsoft.